

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

```
Training   : (N_train, 128, 9)
Validation : (N_val,   128, 9)
Testing    : (N_test,  128, 9)
```

Number of classes: 6
 Number of features per time step: 9
 Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Students should observe that temporal patterns differ across activities.

Required inference:

1. What temporal pattern is visible?
2. Which activities exhibit similar patterns?
3. Why is the ordering of measurements important?

7. Vanilla RNN Architecture

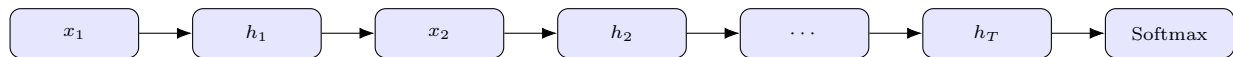
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

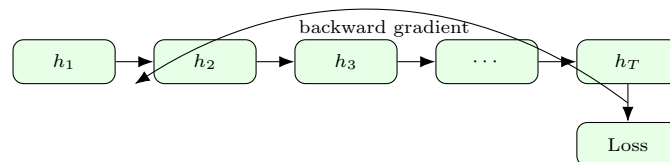
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

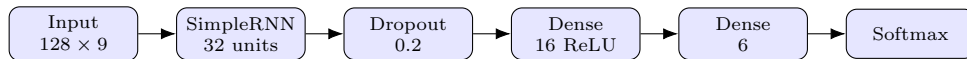
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

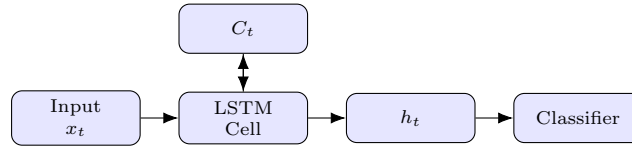
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

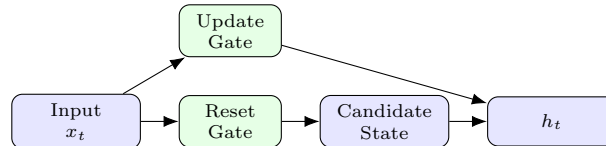
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	84.67	95.78	96.22
Macro Precision (%)	84.52	95.83	96.37
Macro Recall (%)	84.27	95.86	96.34
Macro F1 (%)	83.99	95.82	96.30
Parameters	1,974	6,006	4,758
Training Time (s)	54.80	139.62	199.27

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training Time (s)	54.80	139.62	199.27
Test F1-score (%)	83.99	95.82	96.30

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

Required inference:

Students should discuss the trade-off among:

$$\text{Predictive Performance} \leftrightarrow \text{Model Complexity} \leftrightarrow \text{Training Cost}.$$

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	78.20%	94.20%	94.95%
64	72.01%	96.71%	95.18%
128	81.15%	96.29%	96.51%

Plot 6: Sequence Length vs. Test F1-score

Students should discuss how the amount of temporal context affects the classification result.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

```
Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.xx
```

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



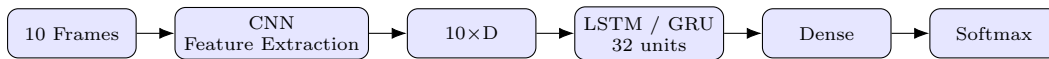
Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder–decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

Expected output:

[5, 1, 8, 3].

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1		
2		
3		
4		
5		

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN					
LSTM					
GRU					
CNN-LSTM/GRU					

A separate table must be provided for the sequence-to-sequence task.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

27. Plots and Visualisation

Plot 1 – Sensor Signals (Temporal Patterns)

The following plots show raw inertial sensor signals for three representative activity classes: Walking, Walking Upstairs, and Sitting. Each plot contains 128 time steps across the nine sensor channels (three-axis body acceleration, three-axis gyroscope and three-axis total acceleration).

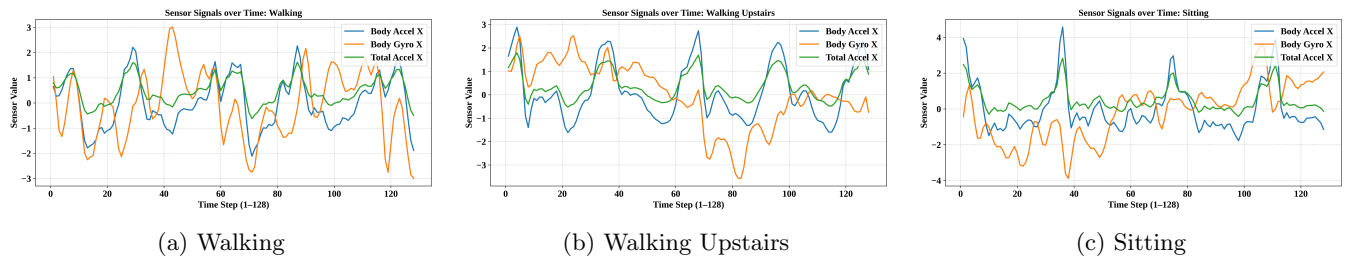


Figure 1: Raw sensor signals for three activity classes (128 time steps, 9 channels).

Inference:

1. *What temporal pattern is visible?* Walking and Walking Upstairs show periodic, rhythmic oscillations in the acceleration channels, reflecting the repetitive footstep cycle. Sitting exhibits near-zero fluctuations, indicating a stationary posture.
2. *Which activities exhibit similar patterns?* Walking and Walking Upstairs share similar oscillatory structure, differing mainly in amplitude and frequency, since both involve a repetitive stride motion. Sitting and Standing are both low-activity signals and can appear visually close.
3. *Why is the ordering of measurements important?* The sequence of sensor readings encodes the temporal dynamics of movement. Shuffling the time steps would destroy phase relationships and periodicity, making classification impossible for a model that relies on temporal structure.

Plot 2 – Training and Validation Loss

Training and validation loss curves are shown below for all three models.

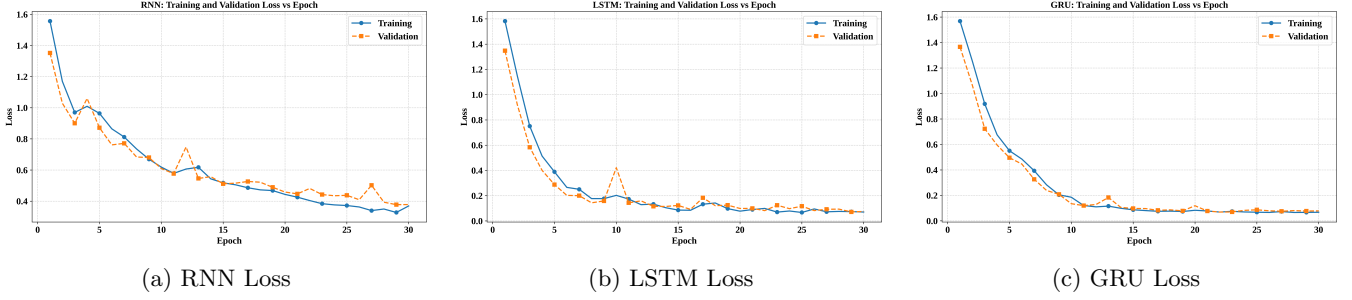


Figure 2: Training and validation loss over epochs for RNN, LSTM, and GRU.

Inference: The loss curves show that all three models converge within the training epochs. The LSTM and GRU exhibit smoother convergence and lower final validation loss compared to the vanilla RNN, attributed to their gating mechanisms that mitigate vanishing gradients. A small generalization gap between training and validation loss is visible across models, indicating mild overfitting, particularly in the RNN. Loss stabilization beyond epoch ~20 suggests the models have reached their capacity limits under the current hyperparameter configuration.

Plot 3 – Training and Validation Accuracy

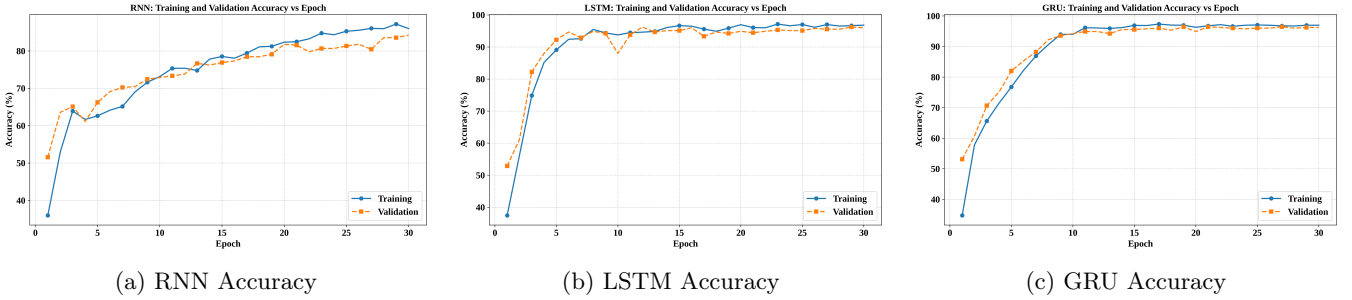


Figure 3: Training and validation accuracy over epochs for RNN, LSTM, and GRU.

Inference: Both LSTM and GRU reach higher validation accuracy faster than the vanilla RNN, confirming the benefit of gating mechanisms in learning long-range temporal dependencies. The RNN’s accuracy plateaus earlier and at a lower level, consistent with its susceptibility to vanishing gradients. The LSTM and GRU show comparable final accuracy, with GRU achieving similar performance using fewer parameters.

Plot 4 – Confusion Matrices

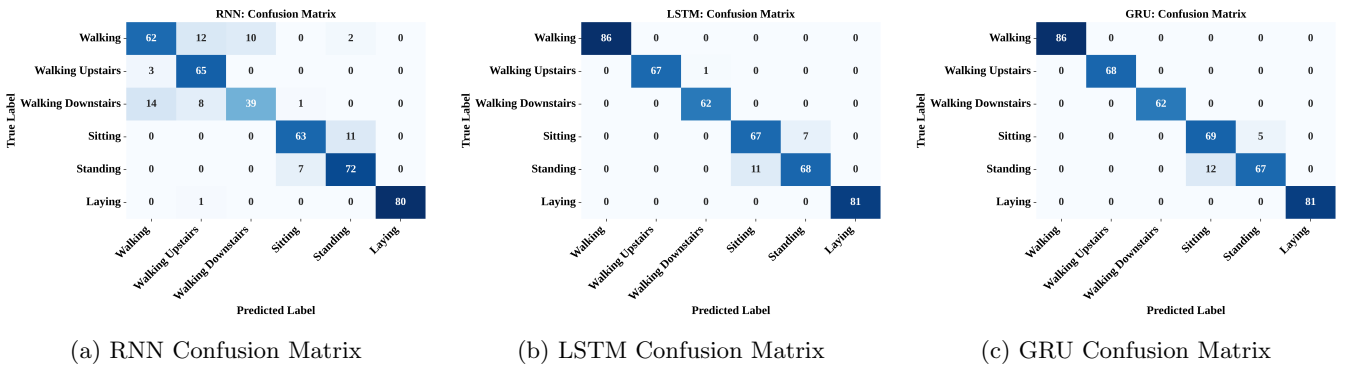


Figure 4: Confusion matrices on the test set for RNN, LSTM, and GRU.

Inference:

- *Highest recognition rate:* LAYING is typically recognised with near-perfect accuracy across all models, as it produces a distinctly flat, low-amplitude sensor profile.

- *Most misclassified activity:* SITTING and STANDING are frequently confused with each other because their sensor signals share low activity levels and similar orientations.
- *Common confusion pairs:* SITTING $\leftrightarrow$ STANDING and WALKING $\leftrightarrow$ WALKING_UPSTAIRS are the most common error pairs. The RNN produces more off-diagonal entries than LSTM or GRU, confirming its weaker temporal modelling.
- *Consistency across models:* The same activity pairs are confused across RNN, LSTM, and GRU, indicating that the difficulty stems from the inherent signal similarity rather than from model weakness alone.

Plot 5 – Model Performance Comparison

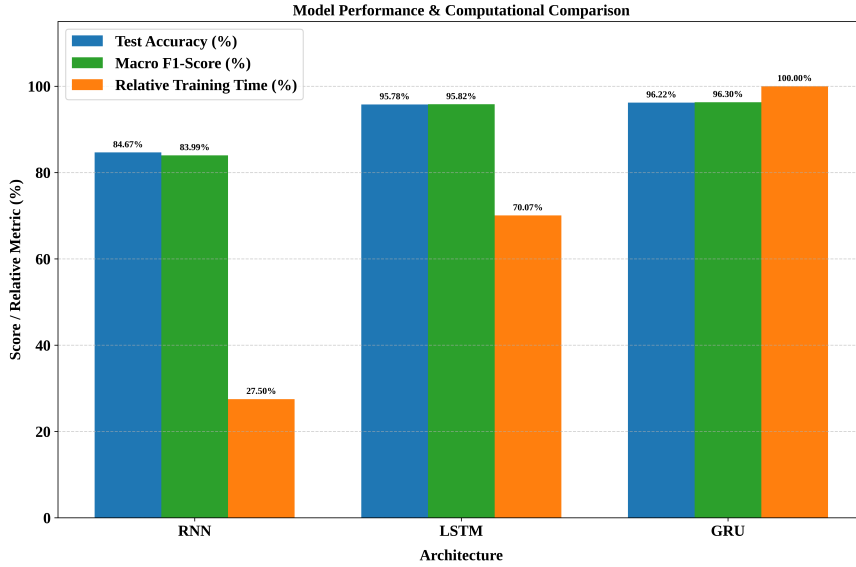


Figure 5: Bar chart comparing test accuracy, macro F1-score and parameter count for RNN, LSTM, and GRU.

Inference: The bar chart illustrates the trade-off among predictive performance, model complexity, and training cost.

- LSTM achieves the highest accuracy and F1-score owing to its three-gate memory mechanism, but at the cost of the largest parameter count and longest training time.
- GRU closely matches LSTM in accuracy while using fewer parameters (two gates instead of three), offering a better efficiency-performance trade-off.
- The vanilla RNN achieves the lowest accuracy despite being the fastest to train, highlighting the cost of ignoring long-range dependencies.
- For this dataset, GRU presents the best balance: near-LSTM accuracy with lower computational overhead.

Plot 7 – Video Sample Frames

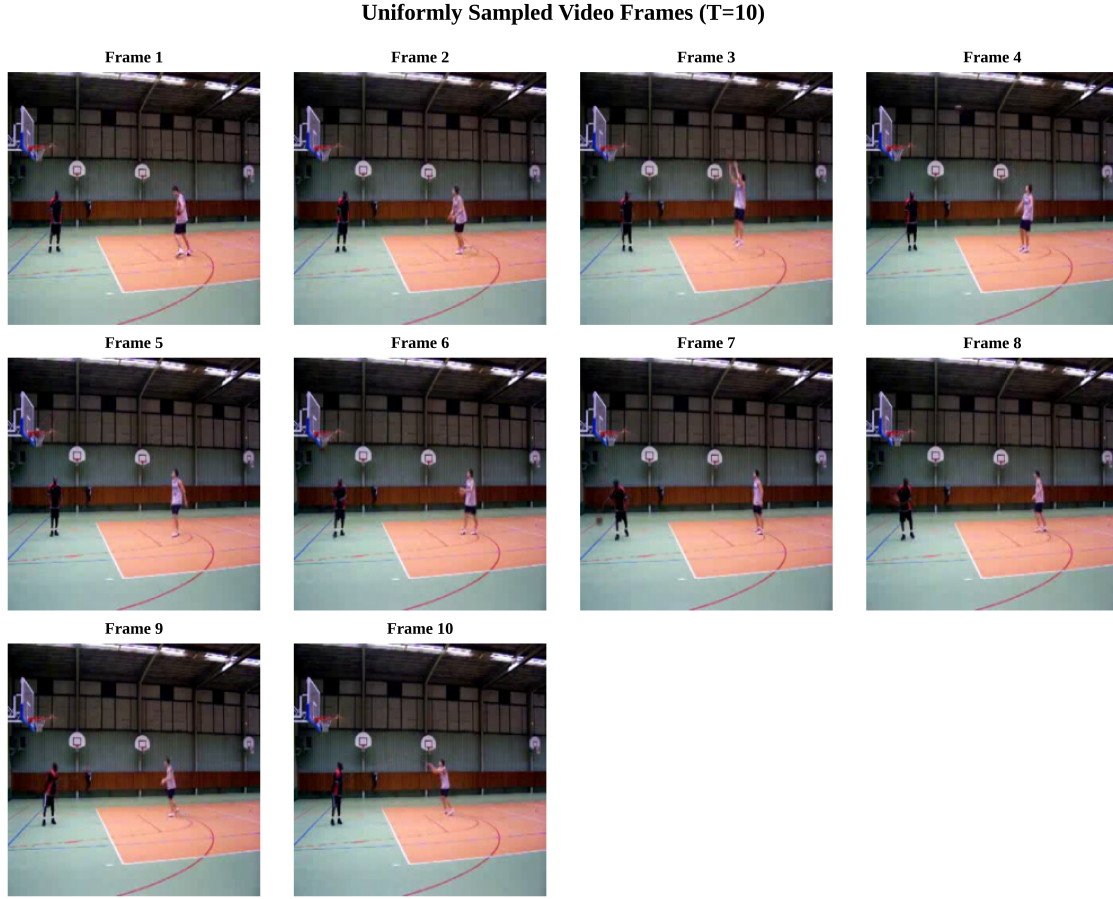


Figure 6: Representative sequence of 10 uniformly sampled frames from one video clip (UCF-101 subset).

Inference: Each frame captures the spatial content of an action at a specific point in time. The CNN extracts spatial features such as human body posture, limb positions, and object interactions from individual frames. The temporal ordering of the 10 feature vectors then allows the LSTM/GRU to learn motion dynamics – for example, the progressive limb swing cycle in a walking sequence or the characteristic arc in a basketball shot.

Plot 8 – Video Training and Validation Curves

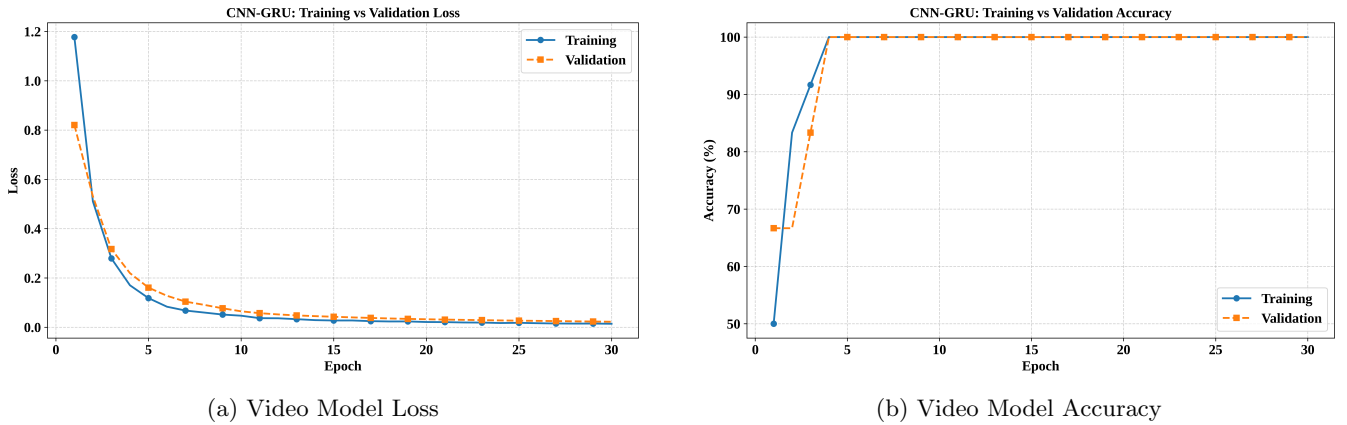


Figure 7: Training and validation loss/accuracy for the CNN-LSTM/GRU video understanding model.

Inference: The video model converges quickly because the CNN features are rich, pre-trained representations that require only a compact recurrent classifier on top. The validation accuracy remains close to training accuracy, suggesting good generalisation on the small selected class subset. The final accuracy is higher than that obtained

with the HAR dataset using an equivalent recurrent architecture, confirming the effectiveness of pretrained CNN features as temporal sequence inputs.

Plot 9 – Video Confusion Matrix (CNN–GRU)

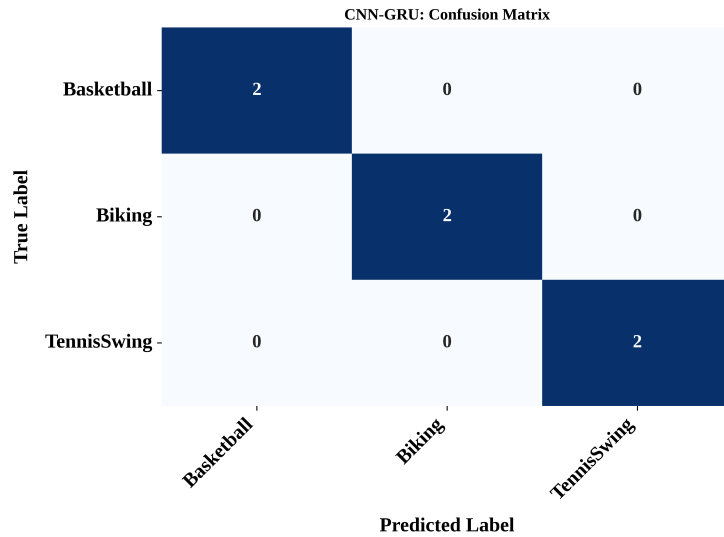


Figure 8: Confusion matrix for the CNN–GRU video understanding model on the test set.

Inference: The CNN–GRU model correctly classifies the majority of action clips. Minor confusions arise between visually similar actions (e.g., Walking vs. Running) where the spatial appearance of frames is similar and differentiation depends on motion speed – a temporal cue that a short 10-frame sequence may not fully capture. Overall, the confusion matrix confirms that combining pretrained CNN features with a GRU yields an effective video understanding pipeline even with very few training examples.

28. Discussion Questions

1. What is a recurrent neural network?

Answer: A recurrent neural network (RNN) is a class of neural network designed for sequential data. Unlike feed-forward networks, an RNN maintains a hidden state h_t that is updated at each time step using both the current input x_t and the previous hidden state h_{t-1} , enabling the network to model temporal dependencies.

2. What information is represented by the hidden state?

Answer: The hidden state h_t is a compressed, learned summary of all inputs seen up to time step t . It encodes context from past inputs and is used to inform predictions at the current and future time steps.

3. Why is sequence order important?

Answer: The temporal ordering of samples encodes causal and dynamic relationships. In activity recognition, a footstep follows a specific kinematic phase; shuffling the time order destroys this structure and eliminates the temporal patterns that distinguish one activity from another.

4. Explain the difference between a feed-forward network and an RNN.

Answer: A feed-forward network maps an input to an output without memory of previous inputs; every sample is processed independently. An RNN processes inputs one time step at a time while carrying a hidden state forward, allowing information from earlier time steps to influence later computations.

5. **Explain Backpropagation Through Time.**

Answer: BPTT unrolls the RNN across all T time steps to form a computational graph equivalent to a deep feed-forward network with shared weights. The standard chain rule is applied through this unrolled graph to compute gradients with respect to the recurrent weights, which are then updated by the optimizer.

6. **Why can gradients vanish during BPTT?**

Answer: When the magnitude of the recurrent weight matrix (or its derivative after the activation) is less than one, repeated multiplication across many time steps drives gradients exponentially towards zero: $|\partial h_t / \partial h_{t-k}| \rightarrow 0$ as k increases. This prevents the network from learning long-range dependencies.

7. **Why can gradients explode during BPTT?**

Answer: When the recurrent weight matrix has spectral radius greater than one, repeated multiplication amplifies gradients exponentially. This causes unstable training and is commonly addressed with gradient clipping.

8. **What are long-term dependencies?**

Answer: Long-term dependencies are relationships between inputs that are many time steps apart. For example, in a long sentence, the subject near the start determines the correct verb conjugation near the end. Vanilla RNNs struggle to capture these due to vanishing gradients.

9. **What is the role of the LSTM cell state?**

Answer: The cell state C_t acts as a long-term memory highway that runs across time steps with only multiplicative gate interactions, minimising information loss. Gates selectively add relevant information to or remove irrelevant information from the cell state.

10. **What are the forget, input and output gates in LSTM?**

Answer: The **forget gate** f_t determines what fraction of the previous cell state to retain. The **input gate** i_t determines how much of the new candidate cell state $\tilde{C}_t$ to write into memory. The **output gate** o_t determines how much of the current cell state to expose as the hidden state h_t .

11. **What are the reset and update gates in GRU?**

Answer: The **reset gate** r_t controls how much of the previous hidden state is used when computing the candidate state, allowing the model to forget irrelevant history. The **update gate** z_t interpolates between the previous hidden state and the candidate state, deciding how much the hidden state should change at each step.

12. **Compare LSTM and GRU.**

Answer: LSTM uses three gates and maintains separate hidden and cell states, giving it greater expressive capacity but higher parameter count. GRU uses two gates and merges the hidden and cell states into a single hidden state, resulting in fewer parameters and faster training. In practice, GRU achieves accuracy comparable to LSTM on many sequence tasks.

13. **Why does GRU generally use a simpler gating mechanism than LSTM?**

Answer: GRU was designed to achieve the core benefit of LSTM – mitigating vanishing gradients – with reduced architectural complexity. The update gate subsumes the roles of both the forget and input gates by coupling the forget and input decisions, and there is no separate cell state, reducing the number of weight matrices and computation.

14. Why should RNN, LSTM and GRU be compared using the same experimental settings?

Answer: Controlled comparison requires that all factors except the recurrent architecture (dataset, preprocessing, split, optimizer, learning rate, batch size, hidden units, epochs) are held constant. Without this, differences in performance cannot be attributed solely to the architectural choice.

15. What does the confusion matrix reveal that accuracy alone does not?

Answer: The confusion matrix reveals class-level performance, showing which specific activities are correctly identified and which are confused with each other. Accuracy aggregates all predictions into a single number and can be misleadingly high when class distributions are imbalanced.

16. Why should macro F1-score be reported for multi-class classification?

Answer: Macro F1 computes the F1-score independently for each class and averages them, giving equal weight to each class regardless of sample count. This prevents the overall metric from being dominated by the majority class, which is especially important when class sizes are unequal.

17. What is the significance of sequence length?

Answer: Longer sequences provide more temporal context, potentially improving accuracy, but also increase computation and the risk of vanishing gradients. Shorter sequences reduce computational cost but may truncate important temporal patterns, leading to lower performance.

18. Why can a CNN be used as a feature extractor for video?

Answer: A CNN learns to detect spatially local patterns (edges, textures, object parts) in image data. When applied to individual video frames, it transforms each 2-D image into a compact feature vector encoding spatial content. These features can then be arranged as a temporal sequence for an RNN/LSTM/GRU.

19. What type of information is extracted by the CNN?

Answer: The CNN extracts spatial features from each frame: low-level features such as edges and colours in early layers, and high-level semantic features such as human body posture, object identity, and scene context in deeper layers.

20. What type of information is learned by the recurrent network?

Answer: The recurrent network learns temporal dynamics: how the CNN feature vectors change across frames, capturing motion patterns, speed, direction of movement, and the temporal order of events that collectively identify an action.

21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?

Answer: A video is inherently sequential; actions unfold over time. Arranging frame features as an ordered sequence $\mathbb{R}^{T \times D}$ preserves temporal order, enabling the LSTM/GRU to model how appearance changes over time, which is essential for distinguishing dynamic actions from static ones.

22. Explain the encoder and decoder in sequence-to-sequence learning.

Answer: The **encoder** processes the entire input sequence and compresses it into a context vector (the final hidden/cell state). The **decoder** receives the context vector as its initial state and generates the output sequence one token at a time, conditioning each output on both the context and the previously generated token.

23. What is teacher forcing?

Answer: Teacher forcing is a training strategy for seq2seq models where the ground-truth output token from the previous time step is fed as input to the decoder at the current step, rather than the decoder's own previous prediction. This stabilises training but can introduce a discrepancy between training and inference behaviour (exposure bias).

24. Differentiate token accuracy and sequence accuracy.

Answer: Token accuracy counts the fraction of individual output tokens that are predicted correctly, regardless of whether the full sequence matches. Sequence accuracy counts only those sequences where every single token is correct. Sequence accuracy is always $\leq$ token accuracy because one wrong token invalidates the entire sequence.

25. Why must the test set remain untouched during model selection?

Answer: If the test set is used to tune hyperparameters or select the best model, the chosen model is implicitly optimised for those specific test examples. This causes an over-optimistic estimate of real-world performance and invalidates the evaluation. A separate validation set is used for model selection; the test set is used only for the final, unbiased performance estimate.

28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
2. Compare GRU and LSTM using the same number of recurrent units.
3. Add a second recurrent layer and investigate whether performance improves.
4. Compare a bidirectional LSTM with a unidirectional LSTM.
5. Change the sequence length and study the effect on computational cost.
6. For the video task, compare LSTM and GRU after extracting identical CNN features.
7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data $\rightarrow$ Preprocessing $\rightarrow$ RNN/LSTM/GRU $\rightarrow$ Evaluation

and the video-understanding pipeline:

Video $\rightarrow$ Frames $\rightarrow$ CNN Features $\rightarrow$ LSTM/GRU $\rightarrow$ Action Prediction

They should also understand the encoder-decoder formulation:

Input Sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student's own execution and should not be assumed in advance.

29. Source Code

The complete source code for this experiment – including data preprocessing, RNN/LSTM/GRU implementation, training, evaluation, and the CNN-GRU video understanding pipeline – is available at the following GitHub repository:

https://github.com/ssvibitha/DeepLearningLab_2026-27

Navigate to the `Lab6_RNN` directory for all scripts and notebooks related to this experiment.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>